SUPER ADMIN

Backend Architecture Documentation

Hostel Management System (HMS)

Module scope: MODULES / SUPER-ADMIN / BACKEND

Architecture design • Integration boundaries • Security • Tenant isolation • Implementation plan

Version 1.0

Target architecture and repository-informed implementation guidance

Super Admin Backend Architecture

Architecture Documentation | Hostel Management System (HMS)

Version: 1.0

Scope: `MODULES/SUPER-ADMIN/BACKEND`

Status: Target architecture and implementation guidance; not a claim that every listed component is already implemented.

1. Purpose

This document defines the backend architecture for the HMS Super Admin module only. It is intended to guide implementation inside `MODULES/SUPER-ADMIN/BACKEND` while integrating with the existing shared Go API, authentication, PostgreSQL database, and platform services.

The Super Admin is responsible for platform-level administration across colleges. Day-to-day hostel operations—such as room allocation, student admission, and hostel operations—remain in college-scoped modules and are outside this module's scope.

2. Architecture goals

- Maintain a clear boundary between platform-wide administration and college-level operations.
- Reuse the repository's existing Go/Gin API, PostgreSQL with `pgx`/`pgxpool`, JWT authentication, and handler-service-repository layering.
- Enforce platform scope and granular permissions for each administrative action.
- Preserve tenant isolation; never trust a client-supplied college ID as proof of access.
- Record security-sensitive administrative actions in an auditable event trail.
- Keep API compatibility with currently registered platform endpoints until the frontend is migrated deliberately.
- Make incomplete features explicit so architecture documentation is not mistaken for implemented production code.

3. Confirmed technology and integration context

- **Backend language/router:** Go with Gin.
- **Database access:** PostgreSQL through `pgx` / `pgxpool`.
- **Authentication:** JWT with explicit `PLATFORM` versus `COLLEGE` scope.
- **Application layering:** handler → service → repository.
- **Shared runtime concerns:** middleware for authentication/authorization, tenant context, request IDs, rate limiting, logging, and recovery under `API/internal/`.
- **Schema/migrations:** `DATABASE/migrations/`.
- **Super Admin frontend:** existing React/Vite package under `MODULES/SUPER-ADMIN/FRONTEND/WEB`.

Do not introduce a separate Node.js backend, duplicate database pool, or new ORM solely for this module.

4. High-level architecture

```
Super Admin Web UI (React/Vite)
            |
            | HTTPS / JSON
            v
Existing Go API / Gin Router
            |
            v
Shared middleware
  - request ID, recovery, logger, CORS, security headers
  - JWT authentication
  - platform-scope enforcement
  - granular permission enforcement
            |
            v
Super Admin route registration / handlers
            |
            v
Super Admin services
  - business rules and workflow validation
  - transaction boundaries
  - authorization context checks
  - audit event creation
            |
            v
Super Admin repositories
  - parameterized SQL via pgx
  - approved platform-wide queries
  - tenant-aware queries when college data is targeted
            |
            v
Existing PostgreSQL schema and migrations
```

The diagram represents the target request path. Existing platform code is currently distributed across `API/internal/server/platform_*`, `API/internal/service/platform_*`, and `API/internal/repository/platform_*`; do not create a second parallel implementation without a controlled migration.

5. Recommended module folder structure

```
MODULES/
■■■ SUPER-ADMIN/
    ■■■ README.md
    ■■■ BACKEND/
        ■■■ README.md
        ■■■ handlers/
        ■   ■■■ dashboard_handler.go
        ■   ■■■ college_handler.go
        ■   ■■■ platform_admin_handler.go
        ■   ■■■ platform_role_handler.go
        ■   ■■■ entitlement_handler.go
        ■   ■■■ college_settings_handler.go
        ■   ■■■ audit_log_handler.go
        ■   ■■■ platform_settings_handler.go
        ■   ■■■ monitoring_handler.go
        ■   ■■■ approval_handler.go
        ■   ■■■ support_access_handler.go
        ■■■ services/
        ■   ■■■ dashboard_service.go
        ■   ■■■ college_service.go
        ■   ■■■ platform_admin_service.go
        ■   ■■■ platform_role_service.go
        ■   ■■■ entitlement_service.go
        ■   ■■■ college_settings_service.go
        ■   ■■■ audit_log_service.go
        ■   ■■■ platform_settings_service.go
        ■   ■■■ monitoring_service.go
        ■   ■■■ approval_service.go
        ■   ■■■ support_access_service.go
        ■■■ repository/
```

```
■   ■■■ dashboard_repository.go
■   ■■■ college_repository.go
■   ■■■ platform_admin_repository.go
■   ■■■ platform_role_repository.go
■   ■■■ entitlement_repository.go
■   ■■■ college_settings_repository.go
■   ■■■ audit_log_repository.go
■   ■■■ platform_settings_repository.go
■   ■■■ monitoring_repository.go
■   ■■■ approval_repository.go
■   ■■■ support_access_repository.go
■■■ models/
■   ■■■ college.go
■   ■■■ platform_admin.go
■   ■■■ role.go
■   ■■■ permission.go
■   ■■■ entitlement.go
■   ■■■ college_setting.go
■   ■■■ platform_setting.go
■   ■■■ audit_log.go
■   ■■■ platform_metrics.go
■   ■■■ approval.go
■   ■■■ support_access_grant.go
■■■ validators/
■   ■■■ college_validator.go
■   ■■■ platform_admin_validator.go
■   ■■■ role_permission_validator.go
■   ■■■ settings_validator.go
■   ■■■ audit_filter_validator.go
■   ■■■ support_access_validator.go
■■■ routes/
■   ■■■ routes.go
■■■ tests/
■   ■■■ college_service_test.go
■   ■■■ platform_admin_service_test.go
■   ■■■ authorization_test.go
■   ■■■ tenant_isolation_test.go
■   ■■■ audit_log_test.go
■   ■■■ entitlement_test.go
■   ■■■ support_access_test.go
■   ■■■ api_acceptance_test.go
■■■ docs/
    ■■■ API_CONTRACT.md
    ■■■ DATABASE_INTEGRATION.md
    ■■■ GAP_ANALYSIS.md
    ■■■ IMPLEMENTATION_PLAN.md
    ■■■ SECURITY_AND_TENANT_BOUNDARY.md
```

Important: this is a target module structure. The uploaded repository did not contain backend source files in `MODULES/SUPER-ADMIN/BACKEND`. Treat these filenames as proposed responsibilities, not existing implemented files.

6. Layer responsibilities

6.1 Routes

- Register Super Admin endpoints with the existing Gin router.
- Apply shared authentication, platform-scope, and granular permission middleware.
- Preserve current paths while existing frontend clients depend on them.
- Keep route definitions declarative; do not put business logic or SQL in route registration.

6.2 Handlers

- Parse and validate HTTP path, query, and JSON body inputs.
- Obtain actor identity and platform authorization context from trusted middleware.

- Call the corresponding service.
- Map domain errors to consistent HTTP responses.
- Avoid embedding business rules or direct SQL in handlers.

6.3 Services

- Enforce business rules and workflow state transitions.
- Authorize sensitive actions at the service boundary where appropriate, in addition to route middleware.
- Coordinate transactions spanning multiple repositories.
- Emit audit events for security-sensitive mutations.
- Apply idempotency or concurrency safeguards to lifecycle and approval operations.

6.4 Repositories

- Own SQL and database mapping.
- Use parameterized queries and context-aware `pgx` calls.
- Keep database transactions controlled by a clear unit-of-work/service boundary.
- Separate platform-wide aggregate queries from college-targeted queries.
- Never construct SQL from untrusted user input.

6.5 Models and validators

- Define request/response DTOs separately from persistence models where useful.
- Validate allowed status transitions, identifiers, pagination, settings keys, effective dates, and role/permission combinations.
- Do not accept trusted actor identity or permission scope from request JSON.

7. Feature boundaries and service ownership

7.1 Platform dashboard and monitoring

Provide cross-college aggregate metrics, college status counts, approved usage indicators, recent administrative activity, and API/database health signals. The currently inspected overview endpoint reports total colleges, active colleges, and pending approvals; additional metrics require implementation.

7.2 College lifecycle management

Support registration, viewing, updating, activation, suspension, deactivation, and onboarding status. Suspension/deactivation should preserve historical records and must not silently delete tenant data.

7.3 Platform administrator and role management

Support platform-scope administrator lifecycle and role assignment. Define granular permissions such as `college.read`, `college.update`, `college.suspend`, `platform_admin.manage`, `role.manage`, `entitlement.manage`, `audit.read`, `settings.manage`, `monitoring.read`, and `support_access.grant`. These are illustrative permission names; align them with the repository's canonical permission naming before implementation.

The inspected authentication service currently supports `SUPER_ADMIN` as a platform role. Delegated platform administrators require a deliberate authentication and RBAC update.

7.4 College module entitlements

Enable or disable modules for a college with actor, reason, effective period, and audit event. Entitlement changes must not grant a user cross-tenant data access by themselves.

7.5 College settings versus global settings

Keep per-college settings separate from global platform configuration. The inspected schema has `platform.college_settings`, which is tenant-specific; a global settings store was not found. Define allowed keys, validation, secret handling, change history, and rollback policy before adding global mutable settings.

7.6 Audit log

Provide immutable, filterable, paginated records including actor, action, timestamp, target resource, affected college (when applicable), request/correlation ID, outcome, and reason where required. The schema has audit protection against update/delete, but the inspected Go source did not show a complete audit reader/writer API layer.

7.7 Approvals and support access

For sensitive or cross-college operational access, require an explicit permission, reason/ticket, target college, narrow scope, expiry, revocation path, and audit event. Use approval or maker-checker workflow where policy requires it. Do not implement a silent platform bypass of college permissions.

7.8 Platform operations

Expose safe status summaries for backup jobs and service health. Do not return secret values or allow arbitrary shell, SQL, backup, or restore commands through ordinary request handlers.

8. Authentication, authorization, and tenant isolation

- Authenticate using the existing JWT middleware.
- Require `PLATFORM` scope for platform-admin endpoints.
- Enforce a specific permission per route/action; platform scope alone is insufficient.
- Derive actor ID and granted permissions from trusted server-side claims/state, not from request payloads.
- For college-targeted configuration, validate the target college and the action's permission.
- Never treat a supplied `college_id` as authorization.
- Keep platform-wide aggregate access distinct from access to identifiable private college records.
- Any exceptional support access must be explicit, time-limited, reason-bearing, revocable, and audited.
- Apply least privilege, rate limiting, secure session/token revocation, and MFA for privileged accounts where supported by the identity design.
- Ensure suspension blocks appropriate access while retaining records needed for audit and legal/operational history.

9. Go package boundary and integration constraint

The shared runtime packages are under `API/internal/`. Go's `internal` import rule prevents packages rooted outside the allowed parent path from importing these packages directly. Therefore, before physically moving runtime code into `MODULES/SUPER-ADMIN/BACKEND`, choose one of these approaches:

- Keep runtime wiring, shared middleware, database pool, and router ownership in `API`; expose a narrow adapter/interface boundary for module code.
- Or perform a planned package-layout refactor so shared packages are public/importable, with tests and imports updated as one controlled change.

Do not move files blindly. The existing router and tests depend on current package locations.

10. API compatibility and endpoint migration

The inspected project currently registers platform endpoints on the root router, including examples such as `/colleges`, `/reports/overview`, and `/approvals/:request_id`. Keep these paths until the connected frontend is migrated and tested. A versioned namespace such as `/api/v1/platform/...` can be introduced as a planned migration rather than an unannounced breaking change.

For every endpoint, document:

- HTTP method and path
- required permission
- request and response schema
- validation rules
- error codes
- audit event behavior
- tenant/platform scope expectations
- pagination and filtering rules

11. Error handling and observability

- Return consistent JSON error responses without exposing SQL, stack traces, tokens, or secret configuration.
- Include request/correlation IDs in logs and error responses.
- Record authorization denials and sensitive administrative actions without logging credentials or unnecessary personal data.
- Monitor latency, error rate, database pool saturation, and dependency health.
- Keep health checks safe and avoid exposing sensitive internals publicly.
- Use structured logs and redact secrets.

12. Testing strategy

- **Unit tests:** service rules, lifecycle transitions, entitlement validation, permission mapping.
- **Repository tests:** SQL behavior against a test PostgreSQL database.

- **Authorization tests:** platform scope and granular permission enforcement.
- **Tenant-isolation tests:** college A cannot read or mutate college B data without an explicit authorized workflow.
- **Audit tests:** every sensitive mutation creates the expected audit record; audit events cannot be changed through normal APIs.
- **Integration tests:** routes, middleware, transactions, and response contracts.
- **Acceptance tests:** end-to-end Super Admin flows with the existing frontend/API paths.

13. Implementation sequence

- Map existing platform handlers, services, repositories, routes, and migration tables to the proposed module responsibilities.
- Agree on the module integration boundary before moving code.
- Add granular permission checks and authorization tests to existing platform routes.
- Complete audit event writing and the read/filter API.
- Implement missing delegated platform-admin authentication/RBAC if required.
- Expand dashboard metrics and monitoring using approved data sources.
- Decide and implement global settings storage separately from college settings.
- Complete support-access grants, expiry, revocation, and audit/approval workflows.
- Add integration and tenant-isolation tests.
- Clear the existing security release gate before production.

14. Known gaps and release gates from repository review

- The Super Admin backend directory is currently a placeholder; target files above are documentation, not implemented code.
- Existing platform features are split among `API/internal/server/platform_*`, `API/internal/service/platform_*`, and `API/internal/repository/platform_*`.
- Existing platform routes generally have platform-scope protection but need granular route-specific permission checks.
- Audit table protections exist, but a complete Go audit-log reader/writer API was not found in the inspected source.
- The dashboard is currently limited to total colleges, active colleges, and pending approvals.
- Delegated platform administrators are not supported by the inspected authentication flow.
- No global settings table was found.
- The repository's Phase 18 reports identify a production blocker pending security audit of `DATABASE/migrations/900_security.sql`.
- The uploaded archive contained a `.env` file despite ignore rules. If it held live credentials, rotate them and keep secrets out of archives/source control.
- Build/test verification could not be rerun in the inspected environment because the required Go `1.26.0` toolchain download was unavailable.

15. Definition of done

The module is ready for release only when:

- all Super Admin routes require platform scope and specific permissions;
- tenant isolation tests pass;
- sensitive mutations produce immutable audit events;
- college lifecycle and entitlements are transaction-safe and retain history;
- support access is explicit, narrow, expiring, revocable, and audited;
- global settings are separated from college settings;
- API contracts match the frontend;
- production security review and required migrations are approved;
- tests pass in the project's supported Go environment.

Appendix A. Responsibility summary

Area	Primary responsibility
Routes	Endpoint registration and middleware
Handlers	HTTP validation and response mapping
Services	Business rules, workflows, transactions, audit
Repositories	Parameterized PostgreSQL access
Models	Domain and persistence representations
Validators	Input and transition validation
Tests	Unit, authorization, tenant isolation, integration

This documentation defines the target architecture and calls out repository findings. It does not claim that proposed files or missing functionality have already been implemented.